# Practical Work 6 - Linked Servers and Distributed Transactions

## Exercise 1 - Combining Access, Excel and text sources

The aim of this exercise is to access three external product sources from SQL Server and combine their rows in a single view. Each source contains the same two fields: `IdProd` and `LibProd`.

### Access database and linked server

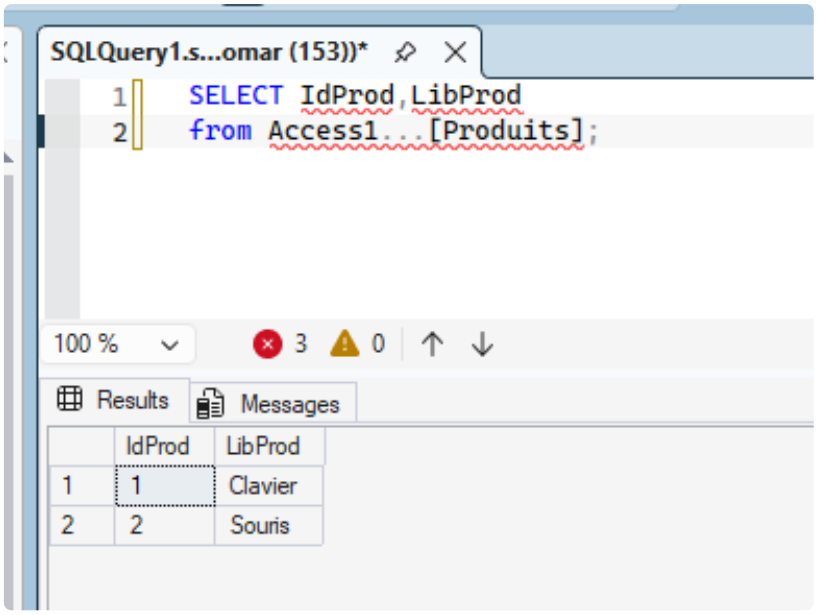
The Access database `C:\TP6\Produits.accdb` was created programmatically through the ACE provider. Its `Produits` table contains two initial products: **1 - Clavier** and **2 - Souris**. This method allowed the database to be prepared with Access Runtime installed.

```
Properties        : System.__ComObject
ConnectionString  : Provider=Microsoft.ACE.OLEDB.16.0;Password="";User ID=Admin;Data
                    Source=C:\TP6\Produits.accdb;Mode=Share Deny None;Extended Properties="";Jet OLEDB:System
                    database="";Jet OLEDB:Registry Path="";Jet OLEDB:Database Password="";Jet OLEDB:Engine Type=6;Jet
                    OLEDB:Database Locking Mode=1;Jet OLEDB:Global Partial Bulk Ops=2;Jet OLEDB:Global Bulk
                    Transactions=1;Jet OLEDB:New Database Password="";Jet OLEDB:Create System Database=False;Jet
                    OLEDB:Encrypt Database=False;Jet OLEDB:Don't Copy Locale on Compact=False;Jet OLEDB:Compact
                    Without Replica Repair=False;Jet OLEDB:SFP=False;Jet OLEDB:Support Complex Data=False;Jet
                    OLEDB:Bypass UserInfo Validation=False;Jet OLEDB:Limited DB Caching=False;Jet OLEDB:Bypass
                    ChoiceField Validation=False
CommandTimeout    : 30
ConnectionTimeout : 15
Version           : 10.0
Errors            : System.__ComObject
DefaultDatabase   :
IsolationLevel    : 4096
Attributes        : 0
CursorLocation    : 2
Mode              : 16
Provider          : Microsoft.ACE.OLEDB.16.0
State             : 1

IdProd  : 1
LibProd : Clavier

IdProd  : 2
LibProd : Souris
```

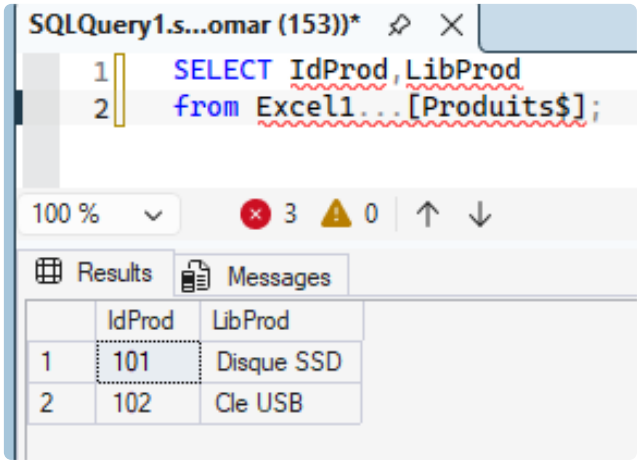
The linked server `Access1` uses `Microsoft.ACE.OLEDB.16.0` to read this database. A query from SQL Server returned both products, confirming access to the external table.



## Excel workbook and linked server

The workbook `C:\TP6\Produits.xlsx` contains the `Produits` worksheet, with **101 - Disque SSD** and **102 - Cle USB**. The linked server `Excel1` uses the ACE provider with Excel workbook properties and a header row.

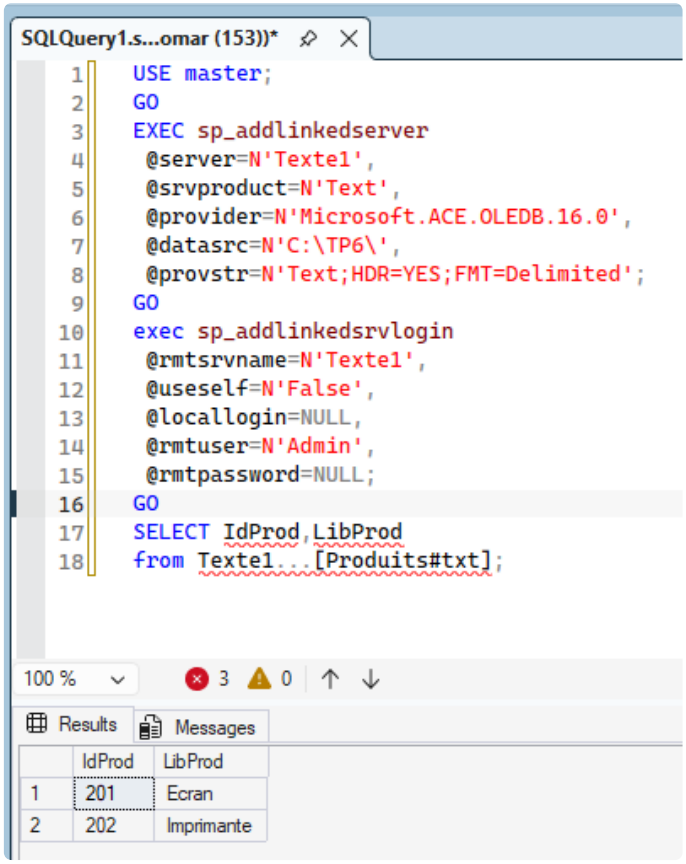
The worksheet is referenced as `[Produits$]` in SQL Server. The query returned the two expected products.




## Text file and linked server

The file `C:\TP6\Produits.txt` contains **201 - Ecran** and **202 - Imprimante**. For a text source, the linked server `Texte1` points to the containing folder. The file is then selected as a table.

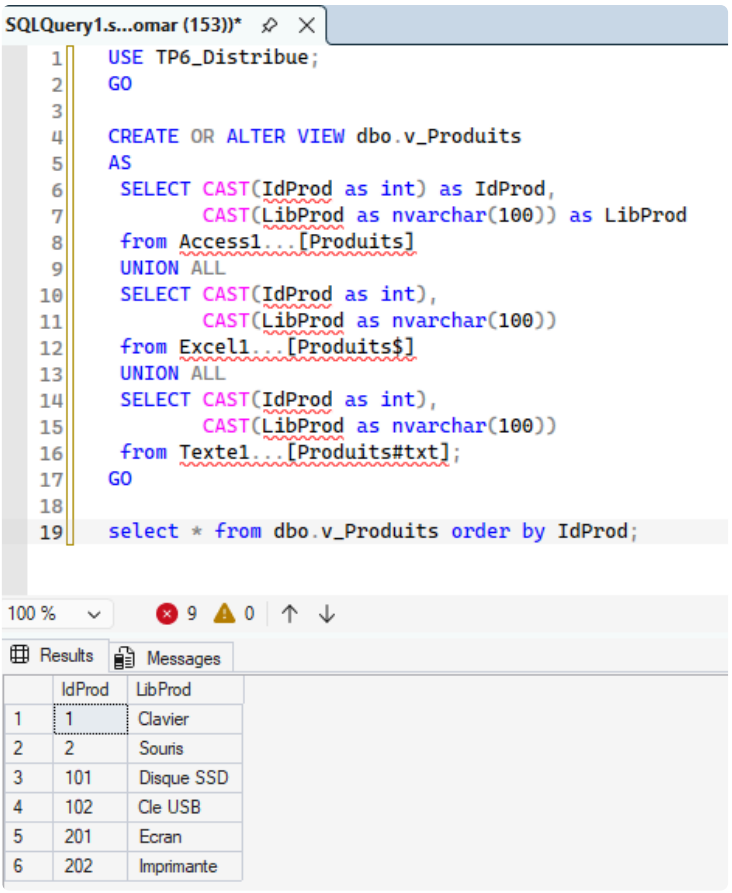
The accompanying `schema.ini` defines the comma delimiter, the header row and the column types. This makes the interpretation of the text file explicit instead of relying on type inference.




## View combining the three sources

The view `TP6_Distribue.dbo.v_Produits` combines the Access, Excel and text sources with `UNION ALL`. The selected columns are aligned as an integer product identifier and a Unicode product name.

The result contains **six rows**, two from each source. `UNION ALL` appends the rows without removing duplicates. This applies the course example of combining external sources through linked servers.




# Exercise 2 - Combining sales from four database sources

The four sources use the same logical structure: `Id`, `Annee`, `Region` and `Montant`. SQL Server uses `int`, `nvarchar(60)` and `decimal(12,2)`. Oracle uses corresponding `NUMBER` and `NVARCHAR2` types; PostgreSQL uses `integer`, `varchar(60)` and `numeric(12,2)`. Conversions in the combined view align the returned values where needed.

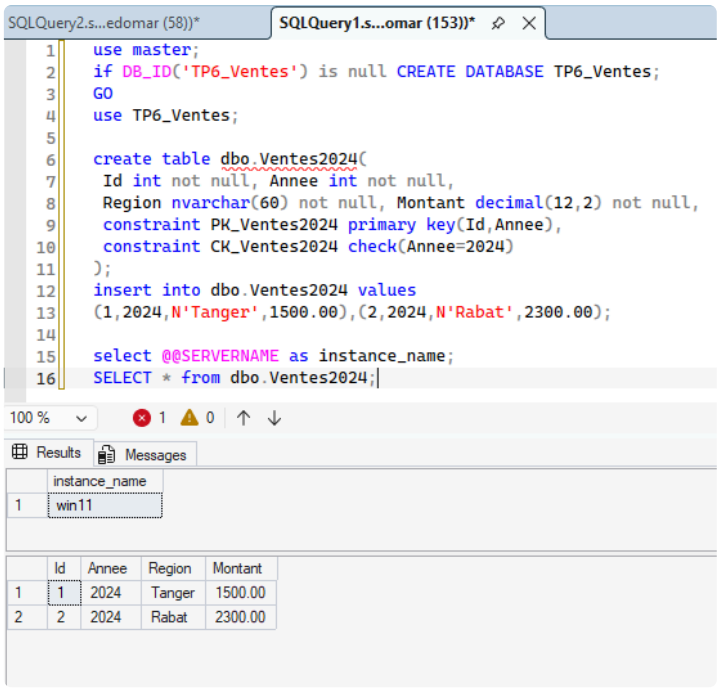
| Source | Location | Sales table | Year |
|---|---|---|---|
| Default SQL Server instance | `WIN11` | `TP6_Ventes.dbo.Ventes2024` | 2024 |
| Named SQL Server instance | `WIN11\ABO` | `TP6_Ventes.dbo.Ventes2025` | 2025 |
| Oracle XE | `XEPDB1` | `TP6.VENTES2023` | 2023 |
| PostgreSQL | `tp6_ventes` | `public.ventes2022` | 2022 |

The existing named instance `WIN11\ABO` is used as the second SQL Server instance requested in the assignment.

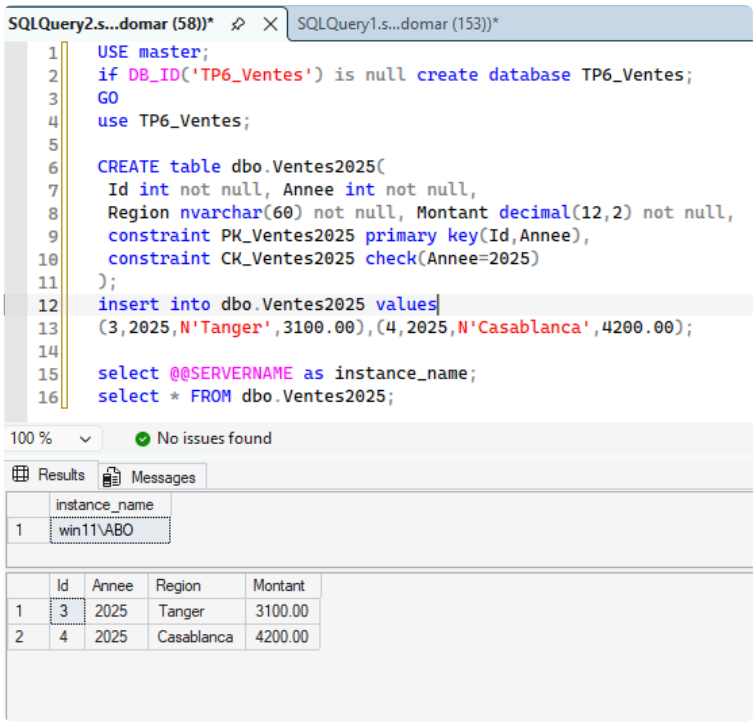
# Question 1 - Configure and test the linked servers

## Second SQL Server instance

The default instance contains `Ventes2024`, with two sales for Tanger and Rabat. Its `CHECK (Annee=2024)` constraint restricts rows to 2024. The primary key contains both `Id` and `Annee`.

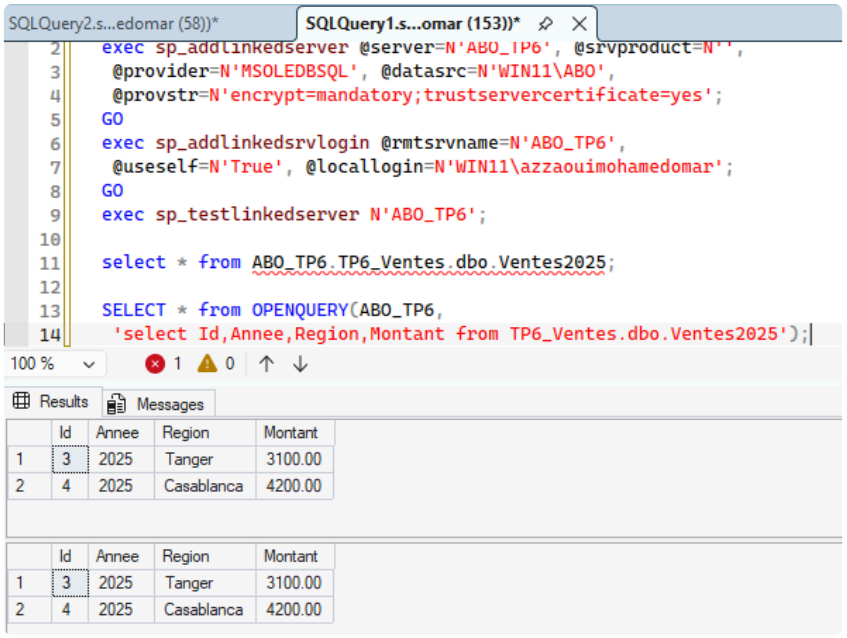



The named instance contains `Ventes2025`, with two sales for Tanger and Casablanca. It has the same column definitions and a corresponding `CHECK (Annee=2025)` constraint.




The linked server `ABO_TP6` connects to `WIN11\ABO` through `MSOLEDBSQL`. Encryption is mandatory. For this local lab, `TrustServerCertificate=yes` allows the connection to use the instance's certificate without validating its certificate chain.

Both a four-part query and `OPENQUERY` returned the same two 2025 rows. These are the linked-server query methods presented in the course.




## Oracle XE

Oracle XE is accessed through its listener at `192.168.11.104:1521`, using the pluggable database service `XEPDB1`. The connection check confirms the selected container.

```
C:\Users\azzaouimohamedomar>sqlplus system@192.168.11.104:1521/XEPDB1

SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 8 09:16:24 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle.  All rights reserved.

Enter password:
Last Successful login time: Sun Nov 08 2026 08:57:32 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> select sys_context('USERENV','CON_NAME') as container_name from dual;

CONTAINER_NAME
--------------------------------------------------------------------------------
XEPDB1
```

The Oracle user `TP6` owns `VENTES2023`. This table contains two sales for 2023: Tanger with an amount of **1800.00**, and Fes with **2600.00**. The rows were committed before testing access from SQL Server.

```
SQL> create table VENTES2023(
  2   Id number(10) not null, Annee number(10) not null,
  3   Region nvarchar2(60) not null, Montant number(12,2) not null,
  4   constraint PK_VENTES2023 primary key(Id,Annee),
  5   constraint CK_VENTES2023 check(Annee=2023)
  6  );

Table created.

SQL> insert into VENTES2023 values(5,2023,N'Tanger',1800.00);

1 row created.

SQL> INSERT into VENTES2023 values(6,2023,N'Fes',2600.00);

1 row created.

SQL> commit;

Commit complete.

SQL>
SQL> column Region format a20
SQL> select * from VENTES2023 order by Id;

        ID      ANNEE REGION                MONTANT
---------- ---------- -------------------- ----------
         5       2023 Tanger                     1800
         6       2023 Fes                        2600
```

The linked server `ORA_TP6` uses `OraOLEDB.Oracle` and maps the local SQL Server login to the Oracle user `TP6`. Four-part access and `OPENQUERY` returned identical results.

```
1   use master;
2   exec sp_addlinkedsrvlogin @rmtsrvname=N'ORA_TP6',
3    @useself=N'False',@locallogin=N'WIN11\azzaouimohamedomar',
4    @rmtuser=N'TP6',@rmtpassword=N'Tp6Lab2026_A';
5   GO
6   exec sp_testlinkedserver N'ORA_TP6';
7
8   select * from ORA_TP6..TP6.VENTES2023;
9   SELECT * from OPENQUERY(ORA_TP6,
10   'select ID,ANNEE,REGION,MONTANT from TP6.VENTES2023');
```

100 % 1 0

Results Messages

| | ID | ANNEE | REGION | MONTANT |
|---|---|---|---|---|
| 1 | 5 | 2023 | Tanger | 1800.00 |
| 2 | 6 | 2023 | Fes | 2600.00 |

| | ID | ANNEE | REGION | MONTANT |
|---|---|---|---|---|
| 1 | 5 | 2023 | Tanger | 1800.00 |
| 2 | 6 | 2023 | Fes | 2600.00 |

## PostgreSQL

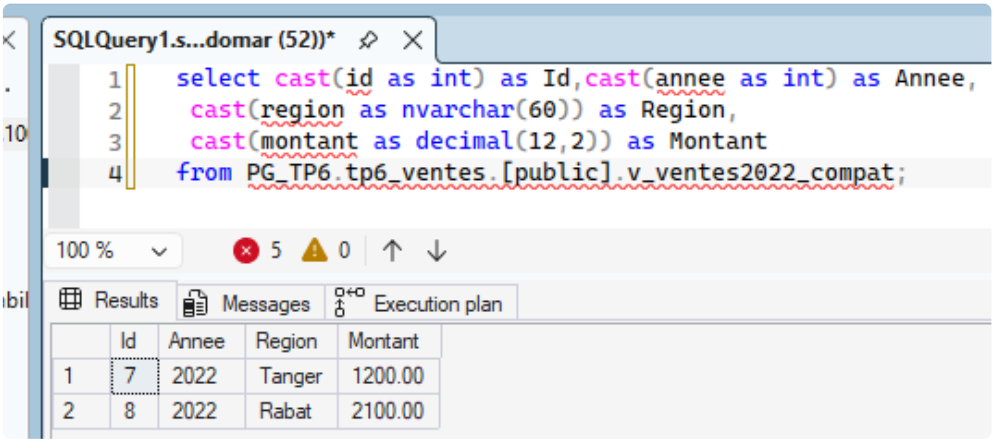
PostgreSQL contains the database `tp6_ventes`, owned by `tp6`. Its `public.ventes2022` table contains **Id 7, Tanger, 1200.00** and **Id 8, Rabat, 2100.00**, both for 2022. A composite primary key and a `CHECK (annee=2022)` constraint define the source table.

```
You are now connected to database "tp6_ventes" as user "tp6".
tp6_ventes=> create table public.ventes2022(
tp6_ventes(>  id int not null, annee int not null,
tp6_ventes(>  region varchar(60) not null, montant numeric(12,2) not null,
tp6_ventes(>  constraint pk_ventes2022 primary key(id,annee),
tp6_ventes(>  constraint ck_ventes2022 check(annee=2022)
tp6_ventes(> );
CREATE TABLE
tp6_ventes=> insert into public.ventes2022 values
tp6_ventes-> (7,2022,'Tanger',1200.00),(8,2022,'Rabat',2100.00);
INSERT 0 2
tp6_ventes=>
tp6_ventes=> SELECT current_database(),current_user;
 current_database | current_user
------------------+--------------
 tp6_ventes       | tp6
(1 row)


tp6_ventes=> select * from public.ventes2022 order by id;
 id | annee | region | montant
----+-------+--------+---------
  7 |  2022 | Tanger | 1200.00
  8 |  2022 | Rabat  | 2100.00
(2 rows)
```

The connection uses a **64-bit PostgreSQL Unicode ODBC driver**, a **System DSN** named `PG_TP6`, and the SQL Server provider `MSDASQL`. The System DSN points to `127.0.0.1:5432` and the database `tp6_ventes`. Access to the original table through `OPENQUERY` returned both rows successfully.

A compatibility view, `public.v_ventes2022_compat`, exposes the four fields as bounded character columns. SQL Server reads that view through a four-part name and casts the values back to `int`, `nvarchar(60)` and `decimal(12,2)`. The test returned the expected rows.



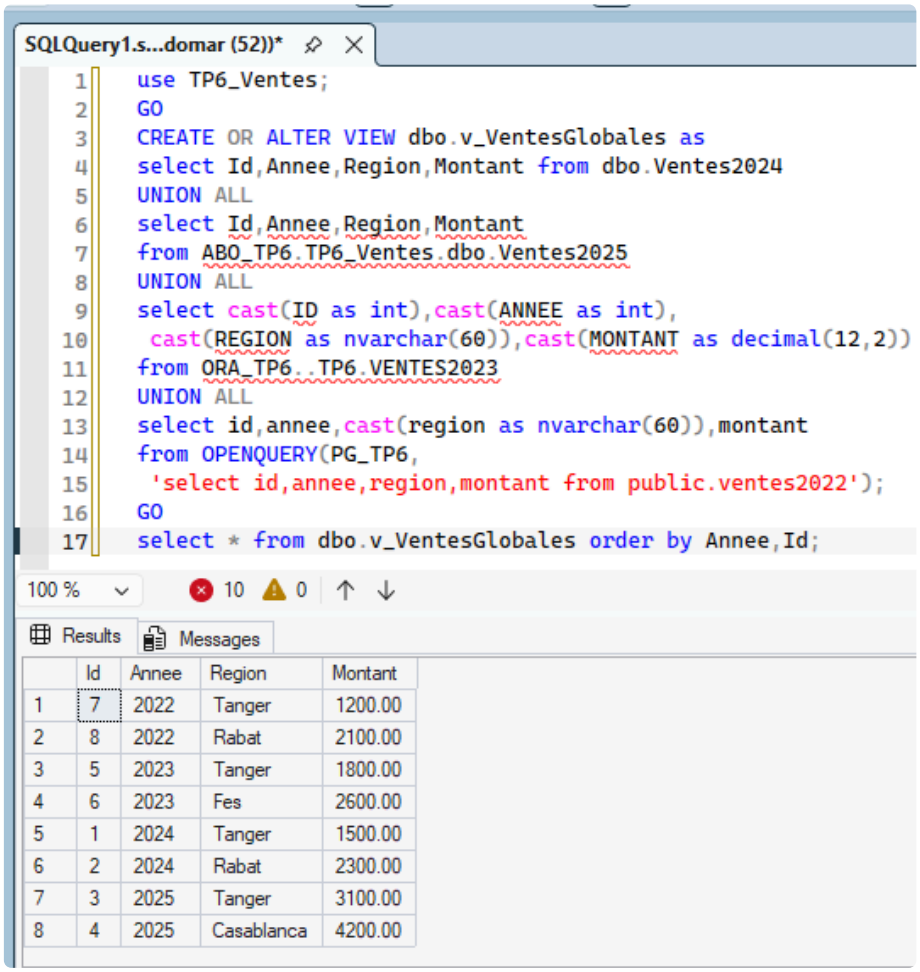

The two successful PostgreSQL tests therefore use different access paths: `OPENQUERY` reads the original table, while four-part access reads the compatibility view.

## Question 3 - Create and test the view combining all four sources

The view `TP6_Ventes.dbo.v_VentesGlobales` combines the four sales sources with `UNION ALL`. It reads the local SQL Server table, the table on `ABO_TP6`, the Oracle table through `ORA_TP6`, and the original PostgreSQL table through `OPENQUERY(PG_TP6, ...)`.

The initial verification returned **eight rows**, two for each year from **2022 to 2025**. This demonstrates a single SQL Server query reading data from the four sources. The later

distributed-transaction test adds two committed test rows; it does not change the validity of this initial result.




# Exercise 3 - Distributed partitioned view and transactions

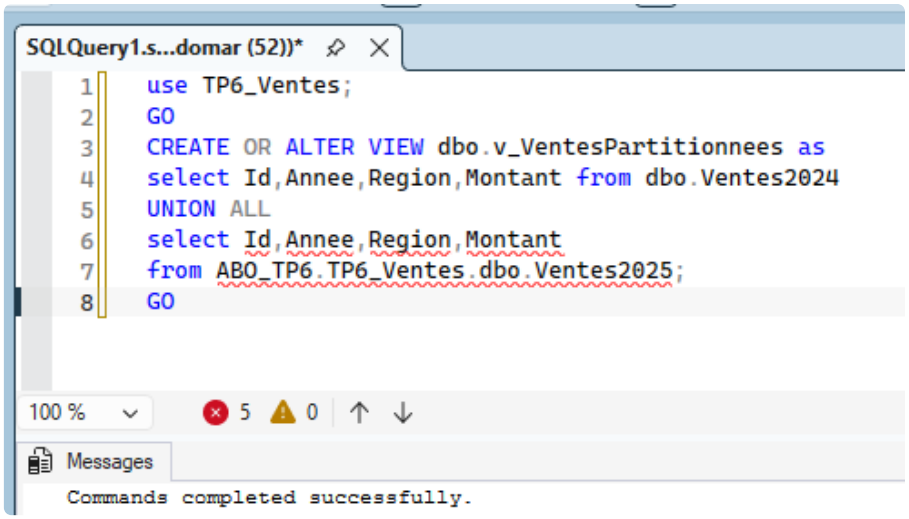
## Create the distributed partitioned view

This exercise explicitly refers to the distributed partitioned-view example in **course pages 17-18**.

The view `dbo.v_VentesPartitionnees` combines only the two SQL Server tables, using `UNION ALL`. Both branches select the same columns in the same order, directly from the underlying tables.

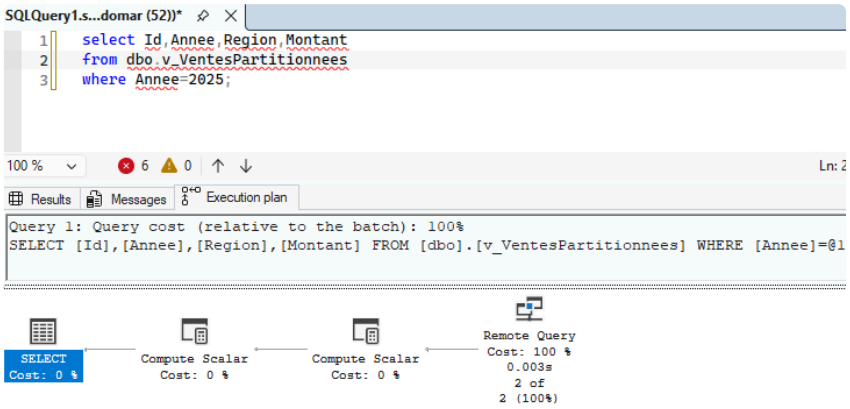
The tables satisfy the main requirements used in the course:

- The `CHECK` constraints define non-overlapping years: 2024 and 2025.
- The partitioning column `Annee` is included in both primary keys.
- The column definitions match between the member tables.
- The member tables have no `IDENTITY` or `timestamp` columns.



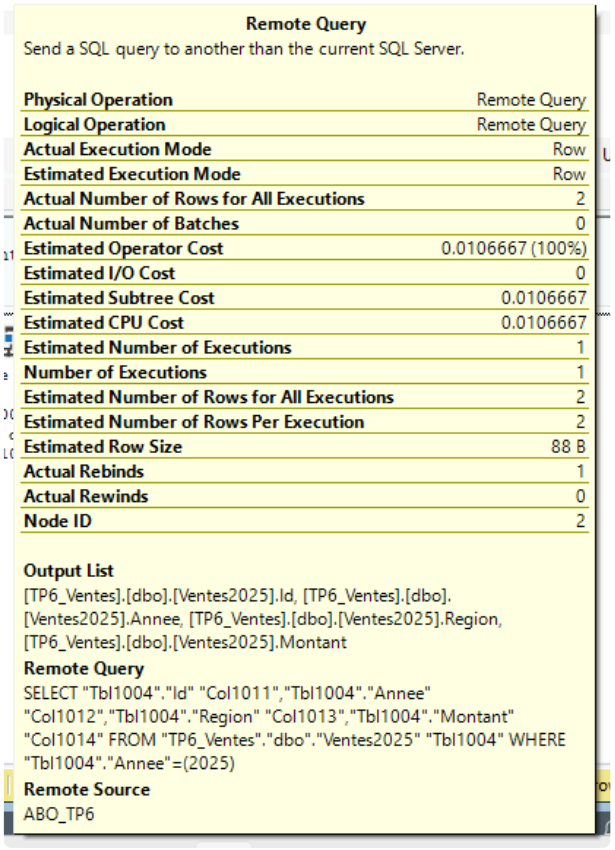

## Filter on 2025 and examine the execution plan

The query with `WHERE Annee=2025` returned the two rows stored on `WIN11\ABO`.




The actual execution plan shows a `Remote Query` with `ABO_TP6` as its remote source. The SQL sent to that source selects from `TP6_Ventes.dbo.Ventes2025` and filters on 2025. The operator returned **two rows**.

The year constraints let the optimizer eliminate the 2024 branch for this query. The data is retrieved from the named instance without scanning the local sales table.




## Question 1 - Modify data through the view

MSDTC was started and network DTC access, inbound transactions and outbound transactions were enabled through Component Services, following **course page 20**.

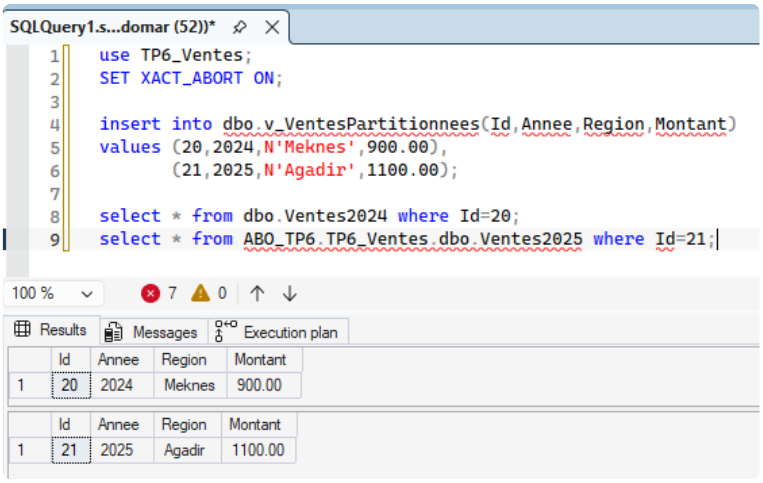
Each modification was executed with `SET XACT_ABORT ON`. Distributed partitioned-view modifications use a distributed transaction to coordinate the affected instances.

### INSERT and verification of row placement

Two test sales were inserted through `dbo.v_VentesPartitionnees`:

| Id | Year | Region | Amount | Destination |
|---|---|---|---|---|
| 20 | 2024 | Meknes | 900.00 | Local `Ventes2024` |
| 21 | 2025 | Agadir | 1100.00 | `Ventes2025` on ABO |

Queries against the member tables confirmed that each row was stored in the table matching its year.




## UPDATE of the sales amounts

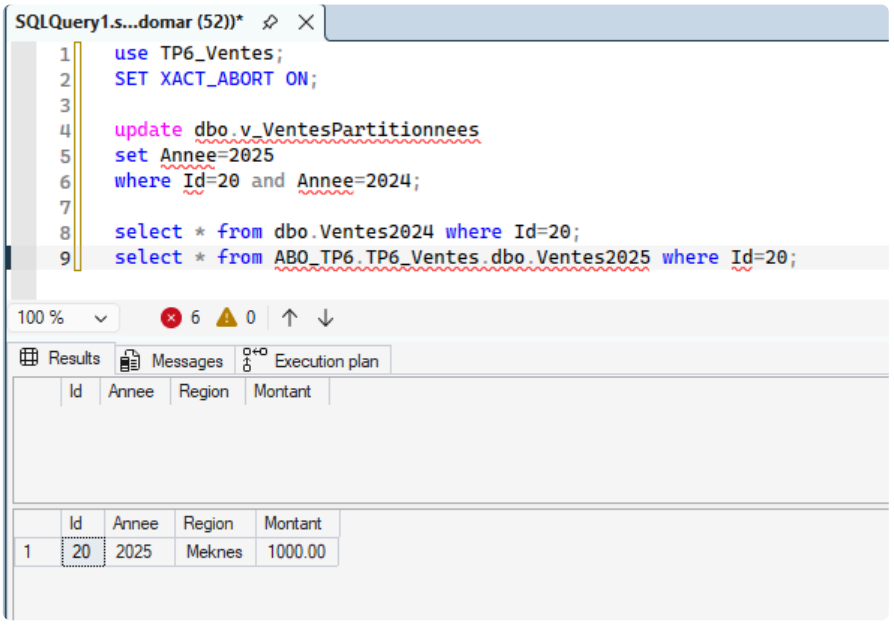
An update through the view added **100.00** to the amount of each test sale. Verification against the two member tables returned **1000.00 for Id 20** and **1200.00 for Id 21**.

This shows that an update issued against the view reaches the corresponding underlying rows on both instances.

## UPDATE changing the year from 2024 to 2025

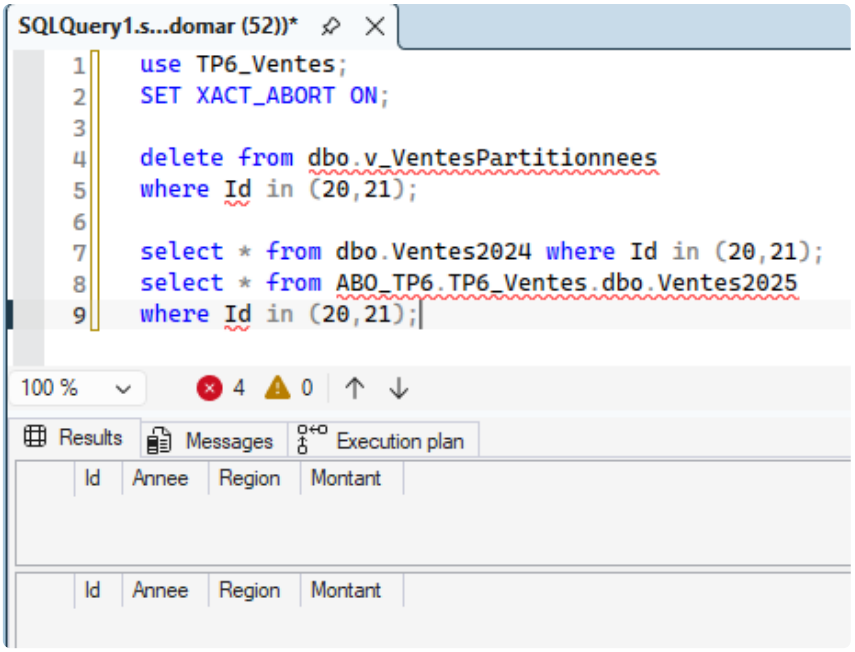
The year of **Id 20** was changed from 2024 to 2025 through the partitioned view. After the update, the local query returned no row for this identifier. The query on ABO returned **Id 20, year 2025, Meknes, 1000.00**.

SQL Server moved the row to the member table whose constraint accepts its new year. The operation is coordinated as a distributed transaction. The year remains part of the primary key, and the destination key must remain unique.




## DELETE and verification

The two test sales, Id 20 and Id 21, were deleted through the view. Queries against both member tables returned empty result sets for these identifiers.




## Question 2 - Distributed transaction between SQL Server and Oracle XE

This test applies the DTC coordination and transaction example from **course pages 19-21**. MSDTC coordinates the transaction, and the Oracle MTS components allow Oracle to participate.

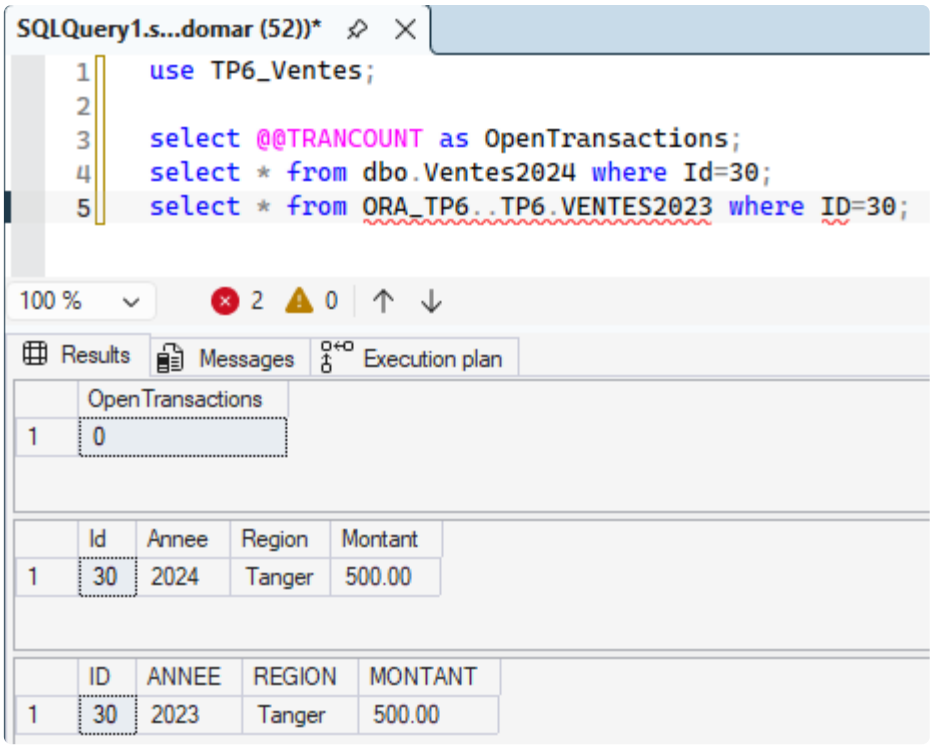
The transaction begins with `BEGIN DISTRIBUTED TRANSACTION` and runs with `SET XACT_ABORT ON`. A `TRY...CATCH` block rolls back any remaining transaction and rethrows the error if execution fails.

### COMMIT case

A test sale with **Id 30**, region **Tanger** and amount **500.00** was inserted into each database:

- SQL Server: `dbo.Ventes2024`, with year **2024**.
- Oracle: `TP6.VENTES2023`, with year **2023**.

After the commit, verification returned the row from both databases. `@@TRANCOUNT` returned **0**, showing that the SQL Server session had no transaction left open.




## ROLLBACK case

A second distributed transaction inserted **Id 31**, region **Rabat** and amount **700.00** into the same two tables, using their respective years. This transaction ended with an explicit `ROLLBACK`.

The final queries returned no row with Id 31 in either database, and `@@TRANCOUNT` was **0**. Both insertions were therefore cancelled. This illustrates the all-or-nothing behavior expected from the distributed transaction.

